

Service-Oriented Architecture (SOA)

An Overview of SOA Architecture

What is Service-Oriented Architecture (SOA)?

- Service-Oriented Architecture (SOA) is an architectural style that focuses on designing software applications as a **collection** of *loosely coupled, reusable services*.
- Each **service** represents a specific **business function** and can be accessed over a network.
- SOA enables *integration, flexibility, and scalability* across various systems.

Parts of SOA

- Services
- Service Registry
- Service Consumer
- Service Provider
- Enterprise Service Bus (ESB)

Services

- These are the **fundamental building blocks** in SOA.
- A service is a **self-contained unit of functionality** that performs a **specific task**, such as data processing, authentication, or transaction handling.

Service Registry

- A directory that stores **information** about the **available services**, their functionalities, and how to interact with them.
- It allows services to be discovered dynamically by other services.

Service Consumer

- Any application, component, or service that **calls or consumes** the **services** provided by other services.

Service Provider

- The service provider is the entity or system that **exposes a service** to other systems or services.

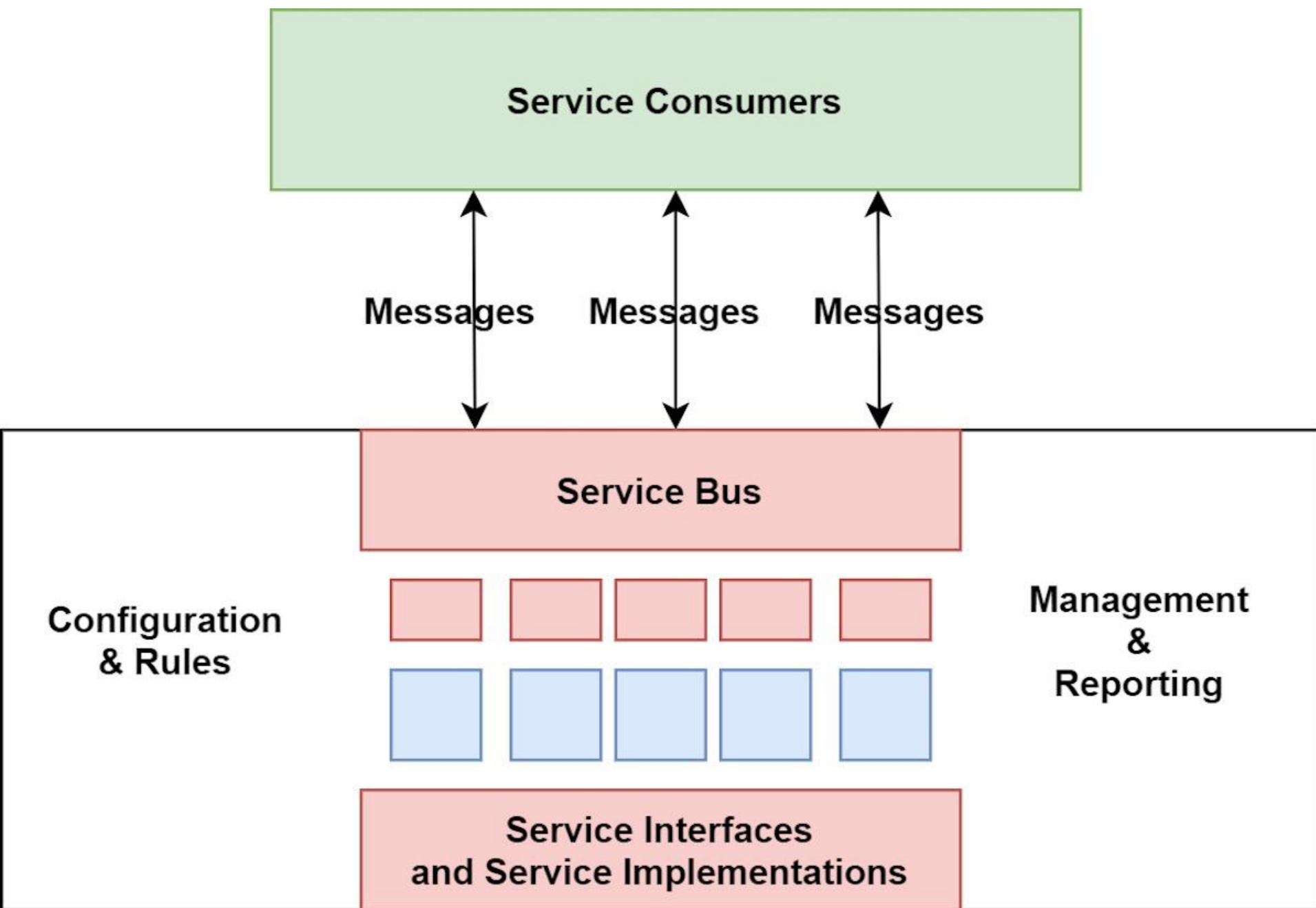
Enterprise Service Bus (ESB)

- A communication layer that facilitates the routing of messages between services.
- The ESB can handle message transformation, routing, and security, and act as the middleware layer for service communication.

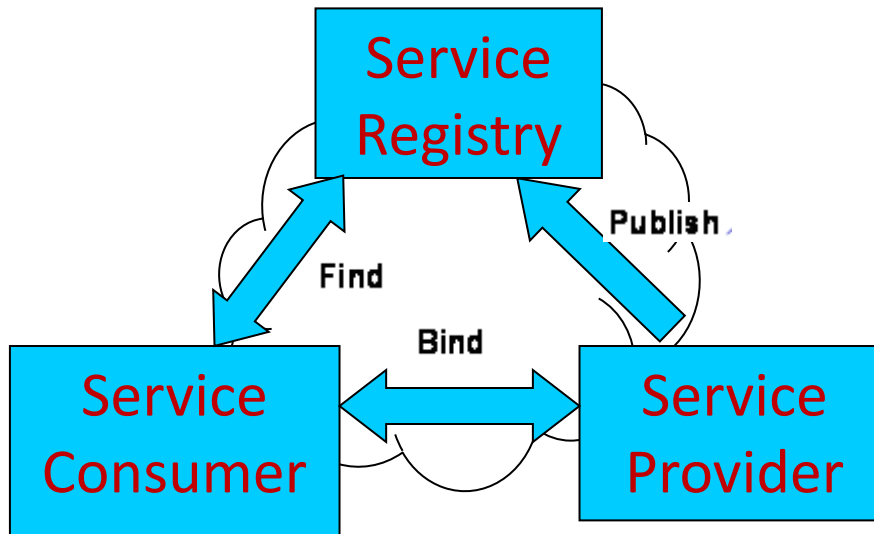
An **Enterprise Service Bus** is an architectural **pattern** and platform that provides a **centralized backbone** for integrating **heterogeneous systems** using **messaging, routing, transformation, and protocol mediation**.

Core Functions of ESB

Function	Description
Message Routing	Directs messages from sender to receiver based on rules (content-based, service-based)
Message Transformation	Converts message formats (e.g., XML to JSON, or one schema to another)
Protocol Mediation	Allows systems using different protocols (HTTP, JMS, FTP, etc.) to communicate
Service Orchestration	Coordinates multiple services into a composite workflow
Security	Handles authentication, authorization, and encryption
Error Handling	Manages retries, dead-letter queues, and fault tolerance
Monitoring & Logging	Tracks performance, usage, and errors



Operations on SOA



Publish: To be accessible, service description must be published so that it can be discovered and invoked by a service consumer.

Find: A service requestor locates a service by querying the service registry for a service that meets its criteria.

Bind and invoke: After retrieving the service description, the service consumer proceeds to invoke the service according to the information in the service description.

Communication Protocols in SOA

- SOA services communicate using standard protocols:
 - **SOAP** (XML-based protocol)
 - REST (HTTP-based architectural style)
 - HTTP/HTTPS (transport protocols)
 - JMS (for asynchronous messaging)
 - CORBA (legacy RPC protocol)

SOAP (Simple Object Access Protocol)

- Messaging protocol used for exchanging structured information in the implementation of web services.
- It allows programs running on different operating systems and written in different programming languages to communicate with each other over a network.
- Characteristics of SOAP
 - Protocol-Independent: SOAP is transport protocol-agnostic, meaning it can operate over a variety of transport protocols. The most common **are HTTP and SMTP**, but it can also be used with others like **FTP, JMS (Java Message Service), or TCP**.
 - XML-Based: SOAP messages are XML-based, which makes them platform- and language-independent. This ensures interoperability between different systems, regardless of the technology or language used to develop them. Since XML is a widely accepted standard, it provides a way for disparate systems to exchange data without worrying about data formatting differences.

SOAP (Simple Object Access Protocol) ctd ...

- Characteristics of SOAP ctd ...
 - Structured Message Format: SOAP messages follow a structured format, consisting of
 - Envelope: The outermost element that identifies the message as a SOAP message.
 - Header: (Optional) Contains metadata or information related to the message, such as authentication credentials, transaction details, or routing information.
 - Body: Contains the actual message data or the request/response payload.
 - Fault: (Optional) Provides error information in case something goes wrong during message processing.
 - Extensibility: SOAP is highly extensible. It supports features such as security (via WS-Security), transaction management, reliable messaging, and more, through various standards and extensions.

SOAP (Simple Object Access Protocol) ctd

```
xml

<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:web="http://www.example.com/webservice">

  <!-- Header (Optional) -->
  <soapenv:Header>
    <web:AuthHeader>
      <web:Username>user123</web:Username>
      <web:Password>password123</web:Password>
    </web:AuthHeader>
  </soapenv:Header>

  <!-- Body (Contains the actual message/content) -->
  <soapenv:Body>
    <!-- Request for a web service method -->
    <web:GetWeather>
      <web:City>New York</web:City>
      <web:Country>US</web:Country>
    </web:GetWeather>
  </soapenv:Body>

  <!-- Fault (Optional, in case of an error) -->
  <!-- This section is included if something goes wrong -->
  <soapenv:Fault>
    <faultcode>soapenv:Client</faultcode>
    <faultstring>Invalid city name</faultstring>
    <detail>
      <errorcode>123</errorcode>
      <errordescription>City name not recognized.</errordescription>
    </detail>
  </soapenv:Fault>

</soapenv:Envelope>
```

REST (Representational State Transfer)

- REST is an architectural style often used for communication in SOA, though it is more common in modern microservices architectures.
- It uses standard HTTP methods like GET, POST, PUT, DELETE to interact with resources (e.g., data objects), and it can use data formats such as JSON or XML.

JMS (Java Message Service)

- JMS is a **messaging protocol** that allows Java-based applications to communicate asynchronously.
- It is often used in SOA for asynchronous messaging and event-driven communication between services.
- JMS is particularly useful when services do not need to be tightly coupled or when **you want to implement message queues for decoupling services.**

HTTP/HTTPS

- HTTP and HTTPS are the underlying protocols for RESTful communication in SOA.
- However, these protocols are not typically used by themselves for communication in a full SOA system (as SOAP or JMS would be).
- Rather, they serve as the transport layer, commonly used to carry SOAP or RESTful messages.

CORBA

- CORBA is a protocol for distributed object communication that was widely used for service integration in **earlier enterprise architectures** (before REST and SOAP became more popular).
- It allows objects written in different programming languages to communicate with each other over a network.
- While CORBA was part of early SOA implementations, it has largely been superseded by SOAP and REST in modern web services.

Examples of SOA

- **Enterprise Resource Planning (ERP) Systems**
 - Integrating modules like finance, HR, and inventory.
- **Banking Systems**
 - Exposing services like transaction processing and customer authentication.
- **E-commerce Platforms**
 - Separating services for product catalog, payment processing, and order management.

How to Divide Services in SOA

- **Business Capabilities:** Define services based on specific business functions (e.g., customer authentication, payment processing).
- **Granularity:** Determine the level of detail required for each service. For example, one service might be responsible for retrieving data, while another service handles complex data manipulation.
- **Reusability:** Design services to be reused across multiple applications or processes.
- **Separation of Concerns:** Ensure each service focuses on a single task or responsibility.

Characteristics of SOA

- **Loose Coupling**
 - Independent services that don't directly depend on each other.
- **Interoperability**
 - Ability to work across different platforms and technologies.
- **Reusability**
 - Services can be reused in multiple applications.
- **Scalability**
 - Each service can be scaled independently.
- **Agility**
 - Enables quick adaptation to changing business needs.

Advantages of SOA

- **Improved Integration**
 - Easier integration of heterogeneous systems.
- **Flexibility & Agility**
 - Ability to quickly adapt and evolve.
- **Cost Efficiency**
 - Reduced development and maintenance costs through reuse.
- **Scalability**
 - Independent scaling of services to meet demand.
- **Vendor Independence**
 - Interoperability with third-party systems, reducing vendor lock-in.

More to read

- Java developers have several options for building and deploying SOAP-based web services,
 - Apache Axis2
 - Spring-WS
 - JAX-WS, the Java API for XML Web Services.

Task

- Write a 1- 2 pages report on
 - The Evolution of Service-Oriented Architecture (SOA) to Microservices Architecture and Its Impact on Modern Enterprises.